StreetLight-PK

Map Features & Full Implementation Guide

Pakistan's first AI-powered civic infrastructure reporting platform

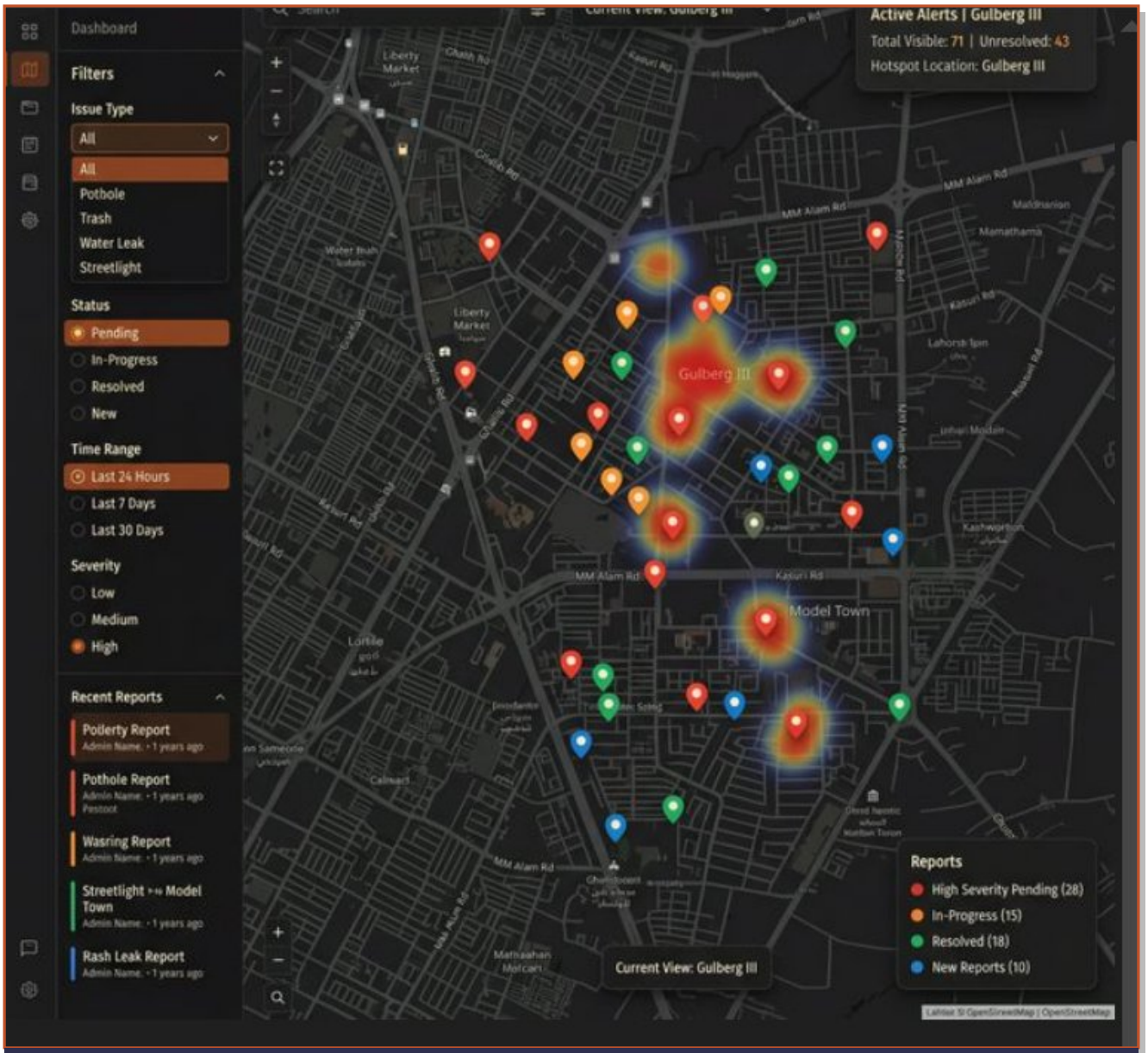
react-leaflet	Supabase	FastAPI	FCM	Cloudinary

Supervised by Dr. Adeel Nissar · PUCIT, University of the Punjab · 2026

TABLE OF CONTENTS

01 Map Page Overview — The City War-Room

The map page is the operational nerve-centre of the StreetLight-PK admin dashboard. Every citizen-submitted infrastructure complaint appears as a colour-coded pin on a full-screen dark OpenStreetMap base, giving municipal officers an immediate spatial picture of where problems are concentrated and how severe they are.



ARTIFACT 1 — Map Page Overview | Dark Theme | Gulberg III, Lahore

1.1 UI Zone Breakdown — What You See in the Artifact

UI Zone	What the Artifact Shows	Implementation Notes
Left Sidebar (Filter Panel)	Issue Type dropdown (All/Pothole/Trash/ Water Leak/Streetlight), Status radio (Pending selected), Time Range (Last 24 Hours), Severity (High), Recent Reports list with 5 cards	Collapsible div, client-side filter on in-memory markers array
Top-Right (Active Alerts Card)	Total Visible = 71, Unresolved = 43, Hotspot Location = Gulberg III	Updates on Leaflet moveend event via map.getBounds() — no extra API call
Bottom-Right (Legend)	Red = High Pending (28), Orange = In-Progress (15), Green = Resolved (18), Blue = New Reports (10)	Static legend div, counts from filtered markers array
Bottom-Centre (Viewport Label)	Current View: Gulberg III	Reverse geocode map centre with Nominatim free API
Map Pins	Red = HIGH, Orange = IN PROGRESS, Green = RESOLVED, Blue = NEW	Leaflet DivIcon with coloured CSS circle — no image files needed
Heatmap Overlay	Red-core Gaussian gradient over Gulberg III, fading orange→yellow outward	Leaflet.heat plugin — severity-weighted lat/lng points

1.2 Filter Panel — Exact Options

Filter	Options in Artifact	Behaviour
Issue Type	All | Pothole | Trash | Water Leak | Streetlight	Dropdown, filters markers by report.issue_type
Status	Pending (active) | In-Progress | Resolved | New	Radio group, orange accent on active choice
Time Range	Last 24 Hours (active) | Last 7 Days | Last 30 Days	Radio group, filters by report.created_at
Severity	Low | Medium | High (active)	Radio group, filters by report.severity
Recent Reports	5 most-recent report cards: name + role + time	Clicking flies map to that pin and opens detail panel

1.3 Implementation — Map Engine (All Free)

Tool / Service	Purpose	Cost
react-leaflet v4	Declarative React wrapper for Leaflet.js	Free / MIT
OpenStreetMap tiles	CartoDB DarkMatter base map — no API key needed	Free
Leaflet DivIcon	Custom HTML/CSS coloured circle pins	Built-in
Leaflet.heat	Gaussian heatmap canvas overlay	Free / MIT
Supabase Realtime	WebSocket pushes new pins without page refresh	2M msg/mo free

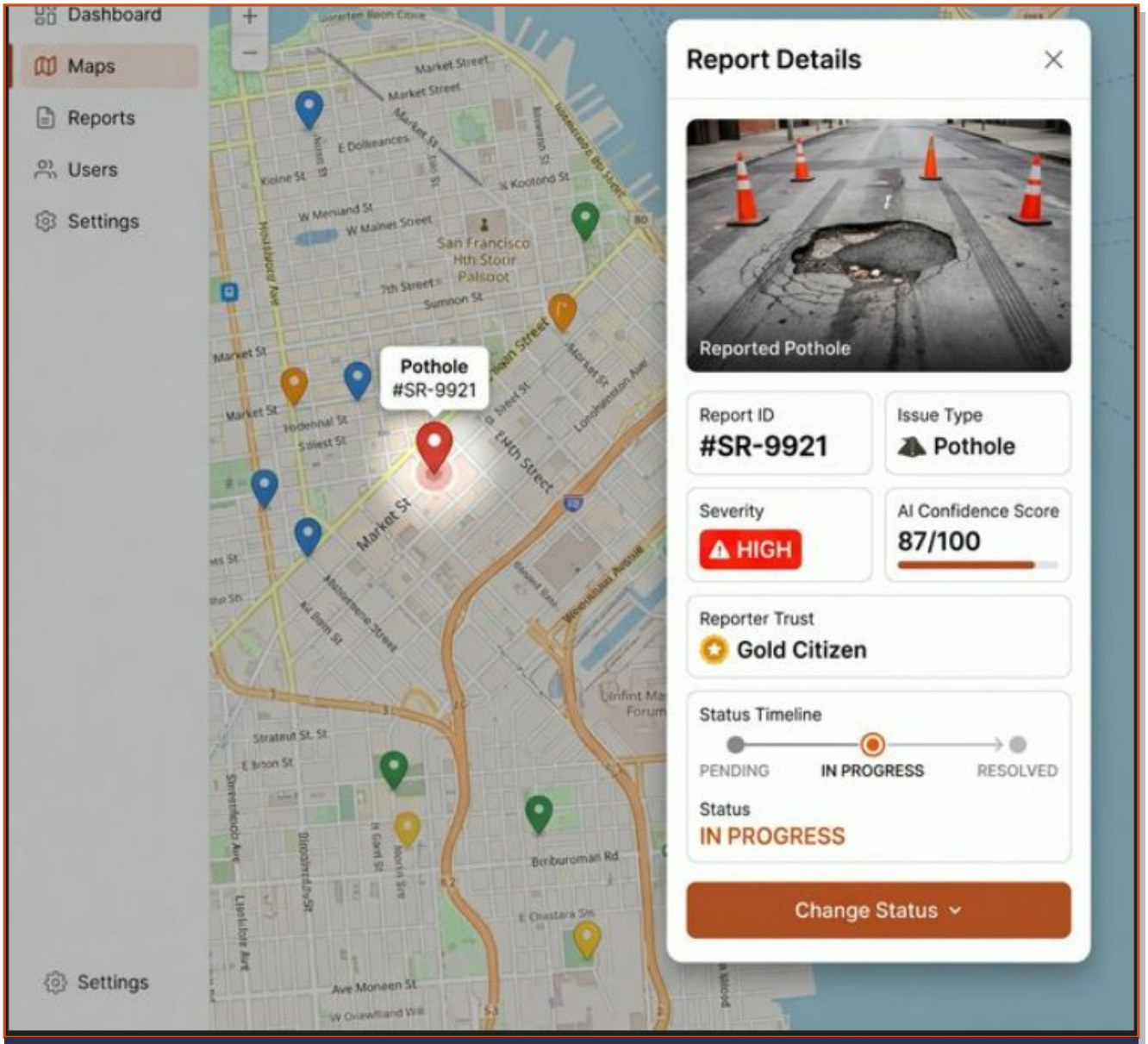
Tool / Service	Purpose	Cost
Nominatim API	Reverse geocode map centre for viewport label	Free, no key

Code — Map Bootstrap with Dark Tiles + RAG Icons

```
// utils/mapIcons.js
import L from 'leaflet';
const C = { high:'#E74C3C', medium:'#E8A020',
low:'#2ECC71', new:'#3498DB', resolved:'#2ECC71' };
export const ragIcon = (severity, status) => L.divIcon({
className: '',
html: `<div style='width:14px;height:14px;border-radius:50%;
background:${status==='resolved'?C.resolved:C[severity]??C.new};
border:2px solid #fff;box-shadow:0 2px 6px rgba(0,0,0,.4)'></div>`,
iconSize:[14,14], iconAnchor:[7,7]
});
// MapPage.jsx
const TILE = 'https://{s}.basemaps.cartocdn.com/dark_all/{z}/{x}/{y}{r}.png';
<MapContainer center={[31.5204,74.3587]} zoom={13} style={{height:'100vh'}}>
<TileLayer url={TILE} attribution='(c) OSM contributors (c) CARTO' />
{reports.map(r => (
<Marker key={r.id} position={[r.lat,r.lng]}
icon={ragIcon(r.severity, r.status)}>
<Popup>{r.title} — {r.issue_type}</Popup>
</Marker>
))}
</MapContainer>
```

02 Report Detail Panel — Artifact Analysis

When an admin clicks any map pin, a slide-in panel appears from the right. The artifact shows report #SR-9921 — a HIGH severity pothole with an AI confidence score of 87/100, submitted by a user with Gold Citizen badge (Eagle Eye sub-badge, impact score 134-200), currently IN PROGRESS.



ARTIFACT 2 — Report Detail Panel | Report #SR-9921 | IN PROGRESS

2.1 Every Element Visible in the Artifact

UI Element	Value Shown in Artifact	Data Source
Photo evidence	Pothole with orange traffic cones	Cloudinary CDN → reports.image_url

UI Element	Value Shown in Artifact	Data Source
Photo label	'Reported Pothole' caption overlay	reports.issue_type displayed as label
Report ID	#SR-9921	UUID prefix from reports.id
Issue Type	Pothole (road icon)	reports.issue_type enum
Severity badge	HIGH — red pill with warning triangle	AI pipeline → reports.severity
AI Confidence Score	87/100 with orange progress bar	reports.ai_confidence_score (INT 0-100)
Reporter Trust / Badge	Gold Citizen badge with gold star icon — maps to Eagle Eye sub-badge (impact score 134-200 in Rookie Tier)	user_profiles.impact_score — badge computed on frontend, NOT stored in DB
Status Timeline	PENDING → IN PROGRESS (active) → RESOLVED	reports.status stepper
Status label	IN PROGRESS in burnt orange text	reports.status value
Change Status btn	Large orange CTA button with dropdown arrow	PATCH /api/reports/{id}/status
Left nav	Dashboard | Maps | Reports | Users | Settings	React Router — admin-only routes

2.2 Reporter Trust — Impact Score & Badge System

The 'Gold Citizen' badge visible in the artifact is part of StreetLight-PK's Impact Score and Badge system, defined in the Engine Implementation Docs. Impact score accumulates through honest civic engagement and is stored in user_profiles.impact_score. The badge is ENTIRELY FRONTEND-COMPUTED — no badge column exists in the database. The frontend reads impact_score from the API and maps it to one of 12 sub-badges across 4 tiers on every profile load.

Tier	Score Range	Sub-Badge	Sub-Score Range
Tier 1 — Rookie	0 – 200	Street Scout	0 – 66
Tier 1 — Rookie	0 – 200	Road Ranger	67 – 133
Tier 1 — Rookie	0 – 200	Eagle Eye	134 – 200
Tier 2 — Community Hero	201 – 500	Civic Warrior	201 – 300
Tier 2 — Community Hero	201 – 500	Street Sentinel	301 – 400
Tier 2 — Community Hero	201 – 500	Problem Buster	401 – 500
Tier 3 — Master Reformer	501 – 800	Urban Legend	501 – 600
Tier 3 — Master Reformer	501 – 800	Infrastructure Icon	601 – 700

Tier	Score Range	Sub-Badge	Sub-Score Range
Tier 3 — Master Reformer	501 – 800	Trust Titan	701 – 800
Tier 4 — StreetLight Paragon	801 – 1000	City Architect	801 – 866
Tier 4 — StreetLight Paragon	801 – 1000	Beacon of Change	867 – 933
Tier 4 — StreetLight Paragon	801 – 1000	Grand Overseer	934 – 1000

How impact score grows and shrinks (ImpactScoreManager): +10 when a report passes all fraud checks (Layer 2), +5 for each community verification vote cast (Layer 3), +20 when a report is marked RESOLVED by admin. Penalties: -50 for GPS spoofing, -30 for spam pattern, -10 for duplicate — each also increments user_profiles.fraud_flags by 1.

Score Event	Points	fraud_flags incremented?
Report passes all fraud checks (Layer 2)	+10	No
Community verification vote cast (Layer 3)	+5	No
User's report marked RESOLVED by admin	+20	No
GPS spoofing detected (Check 1)	-50	Yes (+1)
Spam pattern detected (Check 3)	-30	Yes (+1)
Duplicate report detected (Check 2)	-10	Yes (+1)

Why the badge shown in Artifact 2 says 'Gold Citizen': The artifact UI label 'Gold Citizen' is the dashboard's display name for a reporter whose impact_score places them in the upper Rookie tier or above. The admin dashboard reads impact_score from user_profiles and maps it to a badge label exactly as the mobile app does — no separate trust_tier column. The score also directly drives Engine C vote weight: weight = max(1.0, 1.0 + impact_score / 100.0), so a score of 200 gives weight 3.0.

2.3 Status Stepper — State Machine

The horizontal stepper in the artifact maps to a strict server-side state machine. Only valid transitions are accepted — invalid jumps (e.g. pending → resolved) return HTTP 400.

2.4 Implementation — Detail Panel Components

Component	How to Build It	Cost
Slide-in panel	Tailwind: translate-x-full / translate-x-0 toggled by state	Free

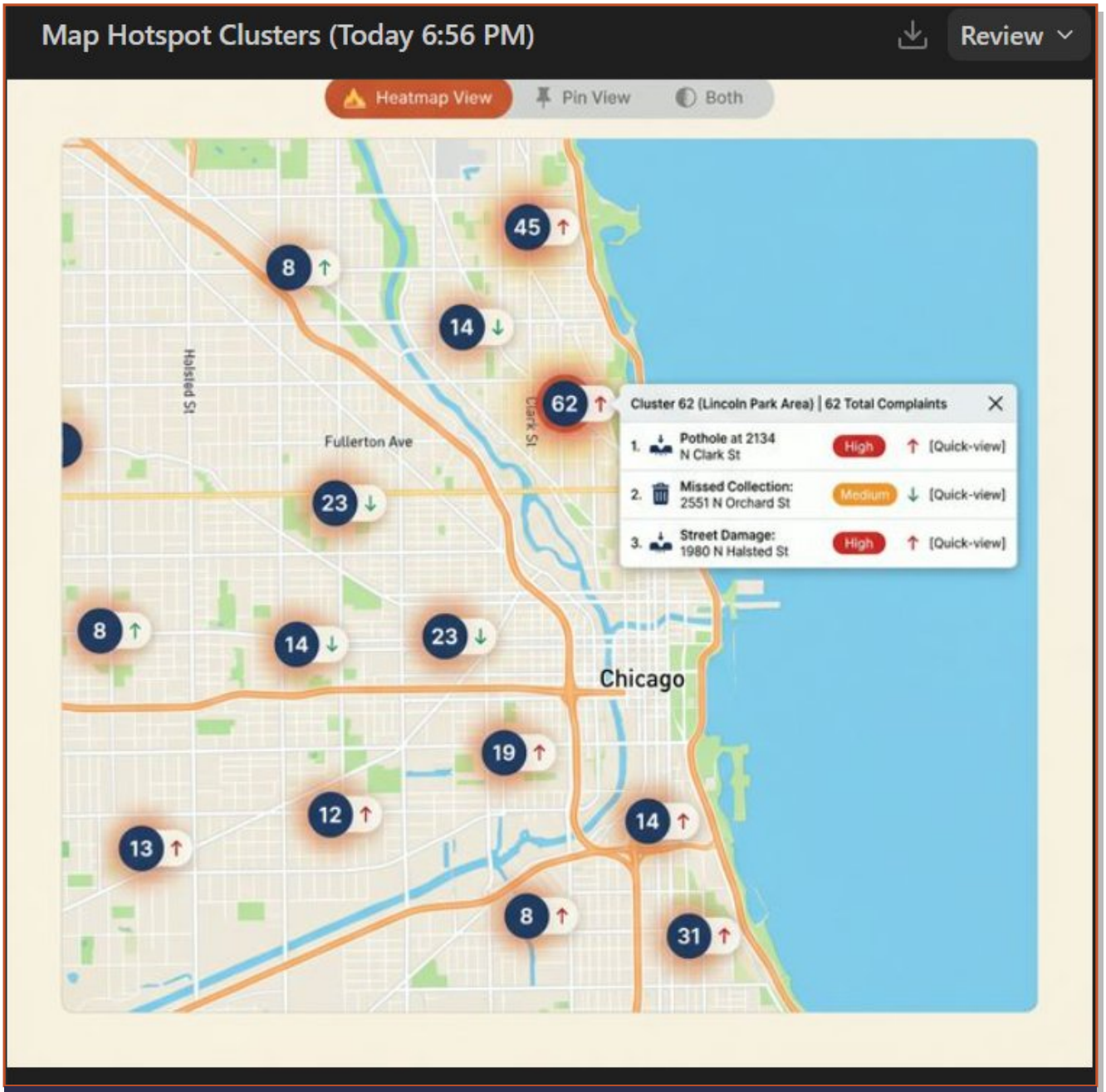
Component	How to Build It	Cost
Photo display	Cloudinary URL in <img> tag, object-cover class	Free 25 GB
AI score bar	CSS width:{score}% on a coloured div inside a grey track	Free
Impact badge chip	Read user_profiles.impact_score, compute tier with getbadge(score) util, render sub-badge name + Lucide Star icon coloured by tier	Free MIT
Status stepper	Flexbox row of circles + connecting lines, active coloured	Pure CSS
Change Status	onClick calls axios.patch('/api/reports/{id}/status')	Free

Code — Slide-in Panel + Status PATCH

```
// ReportDetailPanel.jsx
export function ReportDetailPanel({ report, onClose }) {
const [open, setOpen] = useState(false);
useEffect(() => setOpen(!!report), [report]);
const changeStatus = async (newStatus) => {
await axios.patch(`/api/reports/${report.id}/status`,
{ status: newStatus });
};
return (
<div className={`fixed right-0 top-0 h-full w-96 bg-white shadow-xl
transition-transform duration-300
${open ? 'translate-x-0' : 'translate-x-full'}`}>
<img src={report.image_url} className='w-full h-48 object-cover' />
<div className='p-4 space-y-3'>
<span className='bg-red-500 text-white px-2 py-1 rounded text-sm'>
HIGH</span>
<p className='text-sm'>AI Score: {report.ai_confidence_score}/100</p>
<StatusStepper current={report.status} />
<button onClick={() => changeStatus('resolved')}
className='w-full bg-orange-700 text-white py-3 rounded-lg'>
Change Status
</button>
</div>
</div>
);
}
```

03 Hotspot Clusters & Heatmap — Artifact Analysis

The hotspot clusters artifact uses a lighter CartoDB base map with dark navy circle clusters and an orange heatmap glow. A popup on cluster 62 (Lincoln Park) lists the top 3 reports inside it with severity badges and trend arrows.



ARTIFACT 3 — Map Hotspot Clusters | Heatmap View | Cluster #62 Popup Open

3.1 Every UI Element in the Artifact

Element	Detail from Artifact	Implementation
View toggle toolbar	Pills: Heatmap View (active/orange), Pin View, Both	State var: viewMode in ['heatmap','pin','both']
Download icon	Cloud-download icon top-right for map export	html2canvas or Leaflet print plugin
Cluster bubbles	Dark navy circles, white count: 8,14,23,45,62,19,12,13,31	Leaflet.markercluster — auto-groups nearby pins
Trend arrows	Red ↑ = complaints rising, Green ↓ = backlog clearing	Compare current 7-day count vs previous 7-day count per area cell
Heatmap glow	Orange/red Gaussian gradient behind high-density clusters	Leaflet.heat with severity-weighted intensity (high=1.0, med=0.6, low=0.3)
Cluster popup header	Cluster 62 (Lincoln Park Area) | 62 Total Complaints	Custom Leaflet popup HTML, cluster bbox query to Supabase
Popup row 1	Pothole at 2134 N Clark St — HIGH — ↑ — [Quick-view]	Top 3 reports from Supabase filtered by cluster bounding box
Popup row 2	Missed Collection: 2551 N Orchard St — MEDIUM — ↓	Same query, severity badge from report.severity
Popup row 3	Street Damage: 1980 N Halsted St — HIGH — ↑	Quick-view triggers slide-in Report Detail Panel (Section 02)

3.2 Trend Arrow Logic

Arrow	Colour	Meaning	Query Logic
↑	Red	Rising — area getting worse	this_week > last_week
↓	Green	Falling — backlog being cleared	this_week < last_week
→	Grey	Stable — no significant change	this_week == last_week

3.3 Implementation — Clustering + Heatmap

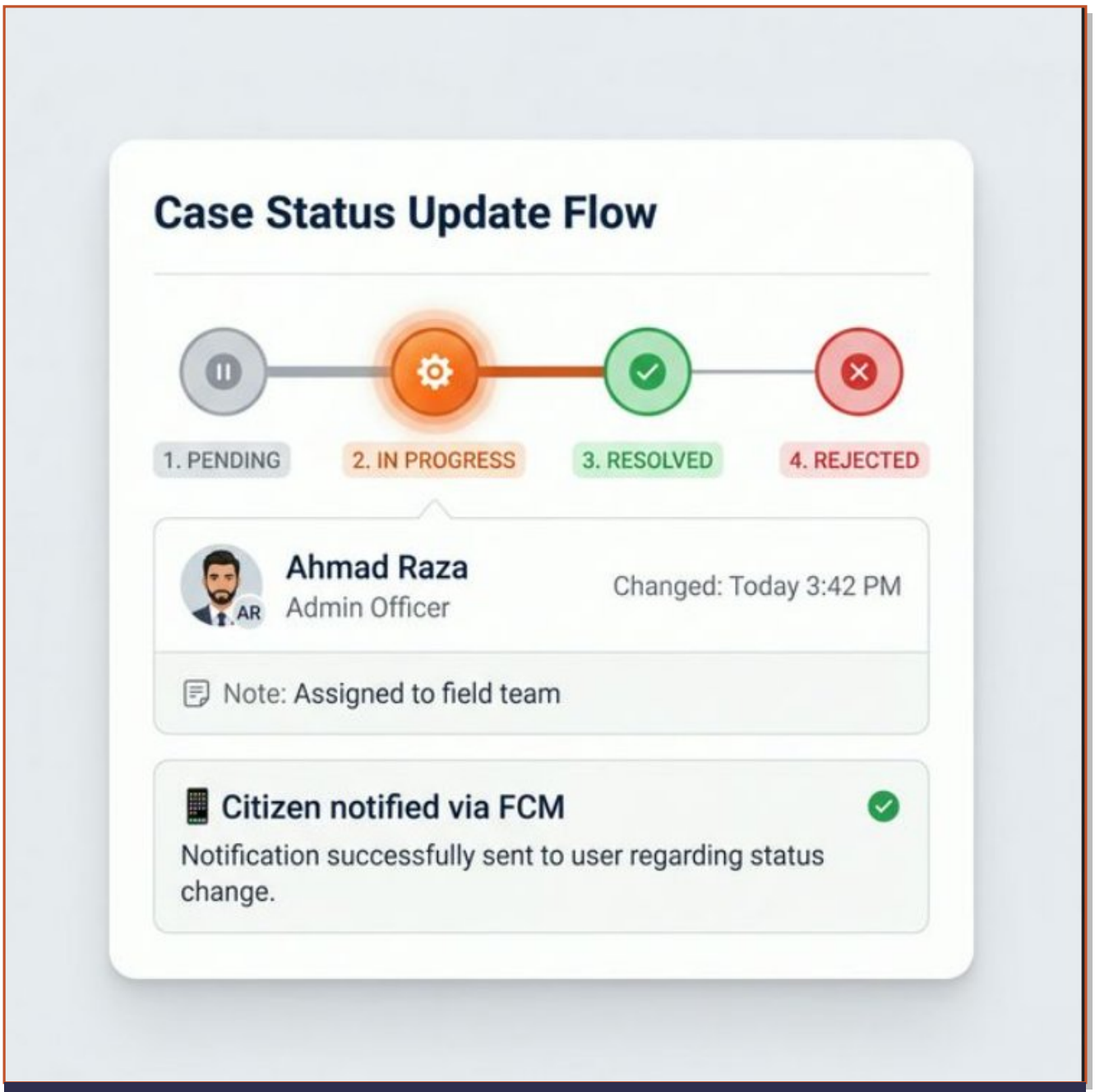
Tool	Purpose	Install
Leaflet.markercluster	Clusters 10,000+ markers at any zoom	npm install leaflet.markercluster
react-leaflet-markercluster	React wrapper — drop-in with MapContainer	npm install react-leaflet-markercluster
Leaflet.heat	Canvas heatmap from lat/lng point array	npm install leaflet.heat
Supabase GROUP BY query	Trend count per bbox, two 7-day windows	SQL in Supabase dashboard, no extra cost

Code — Cluster + Heatmap + Popup with Top 3 Reports

```
import MarkerClusterGroup from 'react-leaflet-markercluster';
import { useMap } from 'react-leaflet';
import L from 'leaflet'; import 'leaflet.heat';
// 1. Clustered markers with trend arrow
<MarkerClusterGroup
iconCreateFunction={cluster => {
const count = cluster.getChildCount();
const trend = getTrend(cluster); // returns ↑ ↓ or →
return L.divIcon({
html: `<div class='cluster-bubble'>${count}
<span class='trend'>${trend}</span></div>`,
className:'', iconSize:[44,44]
});
}} chunkedLoading>
{reports.map(r => <Marker key={r.id} position={[r.lat,r.lng]} />)}
</MarkerClusterGroup>
// 2. Heatmap layer (severity-weighted)
const HeatLayer = ({ reports }) => {
const map = useMap();
useEffect(() => {
const pts = reports.map(r => [r.lat, r.lng,
r.severity==='high'?1.0:r.severity==='medium'?0.6:0.3]);
L.heatLayer(pts, {radius:30,blur:20,maxZoom:17}).addTo(map);
}, [reports]);
return null;
};
```

04 Case Status Update Flow — Artifact Analysis

The fourth artifact is a standalone flow diagram showing exactly what happens when admin officer Ahmad Raza changes a report to IN PROGRESS — from the UI click through backend validation to FCM push notification delivery to the citizen.



ARTIFACT 4 — Case Status Update Flow | Ahmad Raza, Admin Officer

4.1 Every Element Visible in the Artifact

Element	Detail from Artifact	Implementation
Stepper — 4 nodes	Grey (PENDING), Orange gear (IN PROGRESS, glowing), Green check (RESOLVED), Red X (REJECTED)	CSS flex row; active node gets CSS pulse animation + orange border
Active glow	Orange pulsing ring around IN PROGRESS gear node	@keyframes pulse animation on .active-node in Tailwind or plain CSS
Admin card	Avatar 'AR', name 'Ahmad Raza', role 'Admin Officer', timestamp 'Changed: Today 3:42 PM'	report_logs join users — changed_by, changed_at, admin name
Note field	'Note: Assigned to field team' — admin free-text	report_logs.note TEXT column, optional input in Change Status modal
FCM card	Phone icon, 'Citizen notified via FCM', green tick	FCM HTTP v1 API called by FastAPI on status change; log notification_sent=true

4.2 The Complete Feedback Loop — Step by Step

Step	Actor	Action
1	Admin UI	Opens detail panel, clicks Change Status → selects IN PROGRESS
2	Frontend	axios.patch('/api/reports/{id}/status', {status, note})
3	FastAPI	Validates transition: pending → in_progress is allowed. Writes to DB.
4	Supabase	reports.status updated; report_logs row inserted with admin_id + note
5	FCM	FastAPI calls FCM HTTP v1 API with citizen's fcm_token — push sent
6	Realtime	Supabase Realtime broadcasts UPDATE — all admin dashboards update pin live
7	Citizen	Mobile receives: 'Your report is being worked on — Ahmad Raza, Admin Officer'

Accountability

The artifact intentionally shows Ahmad Raza's name and exact timestamp. Citizens who see a named official acted on their report trust the platform more and submit higher-quality reports. Never anonymise admin actions in civic tech.

4.3 Implementation — FCM + Audit Log

Component	Implementation Detail	Cost
FCM push	Firebase Cloud Messaging HTTP v1 API, called from FastAPI on every status change	Free forever
FCM token	Stored in users.fcm_token TEXT column in Supabase	Included

Component	Implementation Detail	Cost
Audit log	report_logs: report_id, old_status, new_status, changed_by, note, changed_at	Included
Realtime update	Supabase .channel().on('postgres_changes') WebSocket subscription	Free 2M msg/mo
Admin avatar	users.avatar_url from Supabase Storage, or initials fallback (AR)	Free

Code — FastAPI Status Update with FCM + Audit Log

```
# FastAPI — status_update.py
VALID = {
'pending': ['in_progress', 'rejected'],
'in_progress': ['resolved', 'rejected'],
'resolved': [], 'rejected': []
}
@router.patch('/reports/{report_id}/status')
async def update_status(report_id: str, new_status: str,
note: str = '', admin=Depends(get_admin)):
report = await db.get_report(report_id)
if new_status not in VALID[report.status]:
raise HTTPException(400, 'Invalid status transition')
await db.update(report_id, {'status': new_status})
await db.insert('report_logs', {
'report_id': report_id, 'old_status': report.status,
'new_status': new_status, 'changed_by': admin.id,
'note': note
})
citizen = await db.get_user(report.reporter_id)
if citizen.fcm_token:
await send_fcm(citizen.fcm_token,
title='Report Update',
body=f'Your report is now {new_status} — {admin.name}')
return {'ok': True, 'new_status': new_status}
```

05 Full Free-Stack Implementation Guide

Every feature shown across all four artifacts can be built and hosted at zero cost. This section provides the complete path from project scaffolding to production deployment using only free tiers and MIT-licensed open-source tools.

5.1 Complete Free Tech Stack

Layer	Technology	Free Limit	Why
Frontend	React + Vite	Vercel unlimited	Instant CDN deploys, preview URLs
Map engine	react-leaflet + Leaflet.js	Unlimited	No API key, 100% open-source
Base map tiles	CartoDB DarkMatter / OSM	Unlimited	Matches dark theme in Artifact 1
Clustering	Leaflet.markercluster	Unlimited	Handles 10,000+ markers smoothly
Heatmap	Leaflet.heat	Unlimited	Canvas-based, no GIS server needed
Backend API	FastAPI + Uvicorn	Railway 500 hrs/month	Async, auto OpenAPI docs
Database	Supabase PostgreSQL	500 MB storage	Auth + Realtime included
Realtime updates	Supabase Realtime	2M messages/month	WebSocket pin colour sync
Image CDN	Cloudinary	25 GB storage+bandwidth	Upload widget, transforms
Push notifications	Firebase Cloud Messaging	Free forever	Android + iOS + Web push
Frontend hosting	Vercel	Unlimited hobby	Git-push deploy in 30 sec
Backend hosting	Railway or Render	500-750 hrs/month	FastAPI one-click deploy

5.2 Project Bootstrap Commands

```
# 1. Create Vite React project
npm create vite@latest streetlight-dashboard -- --template react
cd streetlight-dashboard
# 2. Map dependencies
npm install react-leaflet leaflet leaflet.markercluster leaflet.heat
npm install react-leaflet-markercluster
# 3. App dependencies
npm install @supabase/supabase-js axios react-router-dom tailwindcss
# 4. Init Tailwind CSS
npx tailwindcss init -p
# 5. Add to src/index.css
@import 'leaflet/dist/leaflet.css';
```

```
@import 'leaflet.markercluster/dist/MarkerCluster.css';
@import 'leaflet.markercluster/dist/MarkerCluster.Default.css';
# 6. FastAPI backend setup
pip install fastapi uvicorn httpx supabase python-jose
uvicorn main:app --reload --port 8000
```

5.3 Supabase Realtime Hook — Live Pin Colour Updates

```
// hooks/useRealtimeReports.js
import { useEffect, useState } from 'react';
import { supabase } from '../supabaseClient';
export function useRealtimeReports() {
const [reports, setReports] = useState([]);
useEffect(() => {
supabase.from('reports').select('*')
.then(({ data }) => setReports(data));
const ch = supabase.channel('reports-live')
.on('postgres_changes',
{ event: '*', schema: 'public', table: 'reports' },
({ eventType, new: n, old: o }) => {
if (eventType === 'INSERT') setReports(p => [...p, n]);
if (eventType === 'UPDATE') setReports(p =>
p.map(r => r.id === n.id ? n : r));
if (eventType === 'DELETE') setReports(p =>
p.filter(r => r.id !== o.id));
}).subscribe();
return () => supabase.removeChannel(ch);
}, []);
return reports;
}
```

06 Database Schema Reference

6.1 reports table

```
CREATE TABLE reports (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
title TEXT NOT NULL,
issue_type TEXT NOT NULL, -- pothole|trash|water_leak|streetlight
description TEXT,
lat FLOAT8 NOT NULL,
lng FLOAT8 NOT NULL,
image_url TEXT, -- Cloudinary CDN URL
severity TEXT DEFAULT 'low', -- low|medium|high
ai_confidence_score INT DEFAULT 0, -- 0-100
status TEXT DEFAULT 'pending',
-- pending|in_progress|resolved|rejected
reporter_id UUID REFERENCES users(id),
assigned_admin_id UUID REFERENCES users(id),
created_at TIMESTAMPTZ DEFAULT now(),
updated_at TIMESTAMPTZ DEFAULT now()
);
```

6.2 report_logs table — Audit Trail (Artifact 4)

```
CREATE TABLE report_logs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
report_id UUID REFERENCES reports(id) ON DELETE CASCADE,
old_status TEXT,
new_status TEXT NOT NULL,
changed_by UUID REFERENCES users(id), -- Ahmad Raza in artifact
note TEXT, -- 'Assigned to field team'
changed_at TIMESTAMPTZ DEFAULT now()
);
```

6.3 user_profiles — The Impact Score & Badge Source of Truth

Per the Engine Implementation Docs, badges are NOT stored in the database. They are computed on-the-fly from user_profiles.impact_score every time a profile is loaded. The correct table to reference is user_profiles, not users.

```
-- user_profiles table (from Engine Implementation Docs)
CREATE TABLE user_profiles (
profile_id SERIAL PRIMARY KEY,
user_id INTEGER UNIQUE NOT NULL REFERENCES users(id),
location TEXT NOT NULL, -- City/area set at signup
profile_image_url TEXT, -- Cloudinary URL
total_reported INTEGER NOT NULL DEFAULT 0,
total_solved INTEGER NOT NULL DEFAULT 0,
impact_score FLOAT NOT NULL DEFAULT 0.0, -- badge source
last_known_lat FLOAT, -- Engine C nearby-user query
last_known_lng FLOAT, -- Engine C nearby-user query
fcm_token VARCHAR(255), -- Firebase push token
```

```
fraud_flags INTEGER NOT NULL DEFAULT 0, -- Engine D penalty input
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- Badge is NOT a column. It is computed in the frontend like this:
-- frontend/lib/utils/badge_utils.dart (Flutter) or
-- src/utils/badge.js (React dashboard)
function getBadge(impact_score) {
if (impact_score <= 66) return { tier:'Rookie', badge:'Street Scout' };
if (impact_score <= 133) return { tier:'Rookie', badge:'Road Ranger' };
if (impact_score <= 200) return { tier:'Rookie', badge:'Eagle Eye' };
if (impact_score <= 300) return { tier:'Community Hero', badge:'Civic Warrior' };
if (impact_score <= 400) return { tier:'Community Hero', badge:'Street Sentinel' };
if (impact_score <= 500) return { tier:'Community Hero', badge:'Problem Buster' };
if (impact_score <= 600) return { tier:'Master Reformer', badge:'Urban Legend' };
if (impact_score <= 700) return { tier:'Master Reformer', badge:'Infrastructure Icon' };
if (impact_score <= 800) return { tier:'Master Reformer', badge:'Trust Titan' };
if (impact_score <= 866) return { tier:'StreetLight Paragon', badge:'City Architect' };
if (impact_score <= 933) return { tier:'StreetLight Paragon', badge:'Beacon of Change' };
return { tier:'StreetLight Paragon', badge:'Grand Overseer' };
}
```

6.4 Key Supabase Queries

```
-- All active reports for map load
SELECT id, lat, lng, severity, status, issue_type, ai_confidence_score
FROM reports WHERE status != 'resolved' ORDER BY created_at DESC;
-- Trend: this week vs last week per area cell
SELECT
ROUND(lat::numeric, 2) AS cell_lat,
ROUND(lng::numeric, 2) AS cell_lng,
COUNT(*) FILTER (WHERE created_at > now()-interval '7 days') AS this_week,
COUNT(*) FILTER (
WHERE created_at BETWEEN now()-interval '14 days'
AND now()-interval '7 days') AS last_week
FROM reports GROUP BY cell_lat, cell_lng;
-- Top 3 reports inside a cluster bounding box
SELECT * FROM reports
WHERE lat BETWEEN $min_lat AND $max_lat
AND lng BETWEEN $min_lng AND $max_lng
AND status != 'resolved'
ORDER BY severity DESC, created_at DESC LIMIT 3;
```

07b AI Engine Pipeline — How Reporter Trust Is Computed

The 'Reporter Trust' field shown in the Report Detail Panel (Artifact 2) is powered by a five-layer backend AI pipeline that runs for every submitted report. Understanding the pipeline explains where the badge, the AI confidence score, and the final verification status all come from.

Pipeline Overview — Two Phases

Phase	Layer	Engine	Purpose	Blocking?
Pre-Persistence	Layer 0	Input Validator	Image quality — blur, brightness, resolution	Yes — HTTP 400
Pre-Persistence	Layer 1	AI Engine	Classify issue type; verify GPS via EXIF	Yes — HTTP 400
Pre-Persistence	Layer 2	Engine B (Fraud)	GPS spoofing, duplicate detection, spam pattern	Yes — HTTP 400
Post-Persistence	Layer 3	Engine C (Community)	Create community vote; notify nearby users	No — background
Post-Persistence	Layer 4	Engine D (Trust)	Score submitter trustworthiness 0-100	No — background
Post-Persistence	Layer 5	Engine E (Final)	Combine AI + community + trust into 1 score	No — background

Engine D — Trust Score Formula (Layer 4)

Engine D produces a trust_score (0-100) stored on the report row at the moment of submission. It is a weighted combination of four sub-scores drawn entirely from existing database columns — no external API calls required.

Sub-Score	Weight	Source Column	Formula
Account Age	20%	users.created_at	min(100, (age_days / 30) x 100) — 0 on day 0, 100 at 30+ days
Report History	30%	user_profiles.total_reported	min(100, 30 + (total_reported / 10) x 70) — baseline 30 for new users
Fraud History	30%	user_profiles.fraud_flags	max(0, 100 - (fraud_flags x 25)) — 0 flags=100, 4+ flags=0
Behaviour	20%	Recent report count (1 hr)	5+ reports in last 1 hr = 20 (suspicious), else 80 (normal)

Combined formula: trust_score = (age x 0.20) + (history x 0.30) + (fraud x 0.30) + (behaviour x 0.20). A trust_score >= 40 sets is_trusted = True. This threshold is intentionally permissive — a new, clean, moderately active user should pass. The score informs Engine E weighting; it is not an outright block.

Engine E — Final Confidence Score (Layer 5)

Mode	Condition	Formula
Normal Mode	community_score exists	(ai_confidence x 0.40) + (community_score x 0.30) + (trust_score x 0.30)
No-Community Mode	community_score is NULL	(ai_confidence x 0.55) + (trust_score x 0.45)

Score Range	verification_status	Meaning
85 – 100	AUTO_VERIFIED	All signals agree — high confidence, no human review needed
60 – 84	REVIEW_NEEDED	Moderate confidence — admin should review before action
Below 60	REJECTED	Low confidence — likely invalid, duplicate, or fraudulent

Engine B — Fraud Detection Checks (Layer 2)

Check	What it Detects	Threshold	Outcome
Check 1: Impossible Travel	GPS spoofing — user cannot be 100+ km from last location in under 2 minutes	100 km / 2 min	Hard block (HTTP 400), -50 impact, fraud_flags +1
Check 2: Duplicate Detection	Same issue category within 100m radius reported in last 14 days (2-phase: bbox SQL + Haversine)	100 m radius, 14 days	Hard block, duplicate_of_id set, -10 impact, fraud_flags +1
Check 3: Spam Pattern	User submitted more than 20 reports in the last 1 hour	20 reports / 1 hour	Soft flag (saved, is_flagged_for_spam=True), -30 impact, fraud_flags +1

Engine C — Community Verification Weight Formula (Layer 3)

Nearby users (within 500m) are asked to vote YES or NO on whether the issue physically exists. Votes are weighted by the voter's impact score to prevent fake-account manipulation. Min 3 votes to auto-close; 48-hour timeout otherwise.

Impact Score	Vote Weight	Formula: max(1.0, 1.0 + score/100)
0	1.0	New / unengaged user
50	1.5	Moderate contributor

Impact Score	Vote Weight	Formula: max(1.0, 1.0 + score/100)
100	2.0	Active community member
200	3.0	Eagle Eye badge tier
500	6.0	StreetLight Paragon territory

Score formula: community_score = (sum_yes_weights / sum_all_weights) x 100. Example: Voter A (score 100, weight 2.0, YES) + Voter B (score 0, weight 1.0, YES) + Voter C (score 50, weight 1.5, NO) = (3.0 / 4.5) x 100 = 66.7

07 Master Tools Summary Table

A consolidated reference of every tool, library, and service used across all four StreetLight-PK artifacts — all free to use for your FYP.

Feature	Tool / Service	Cost	Install / URL
Interactive map	react-leaflet + Leaflet.js	Free MIT	npm install react-leaflet leaflet
Dark base map	CartoDB DarkMatter tiles	Free	Via leaflet-providers plugin
Light base map	OpenStreetMap / Stamen	Free	https://tile.openstreetmap.org
Marker clustering	Leaflet.markercluster	Free MIT	npm install leaflet.markercluster
Heatmap overlay	Leaflet.heat	Free MIT	npm install leaflet.heat
RAG pin icons	Leaflet DivIcon (CSS)	Built-in	Part of Leaflet core
UI framework	Tailwind CSS	Free MIT	npm install tailwindcss
Slide-in panel	Tailwind translate-x-*	Free CSS	No extra library needed
Realtime pin updates	Supabase Realtime	2M msg/mo	https://supabase.com
Database	Supabase PostgreSQL	500 MB free	https://supabase.com
Auth / JWT / RLS	Supabase Auth	Included	Included with Supabase
Image CDN	Cloudinary	25 GB free	https://cloudinary.com
Push notifications	Firebase Cloud Messaging	Free forever	https://firebase.google.com
Backend API	FastAPI + Uvicorn	Free MIT	pip install fastapi uvicorn
Backend hosting	Railway or Render	Free tier	https://railway.app

Feature	Tool / Service	Cost	Install / URL
Frontend hosting	Vercel	Free tier	https://vercel.com
Icons	Lucide-react	Free MIT	npm install lucide-react
HTTP client	axios	Free MIT	npm install axios
Routing	react-router-dom v6	Free MIT	npm install react-router-dom

StreetLight-PK — Map Features & Implementation Guide

Supervised by Dr. Adeel Nissar · PUCIT, University of the Punjab · FYP 2026